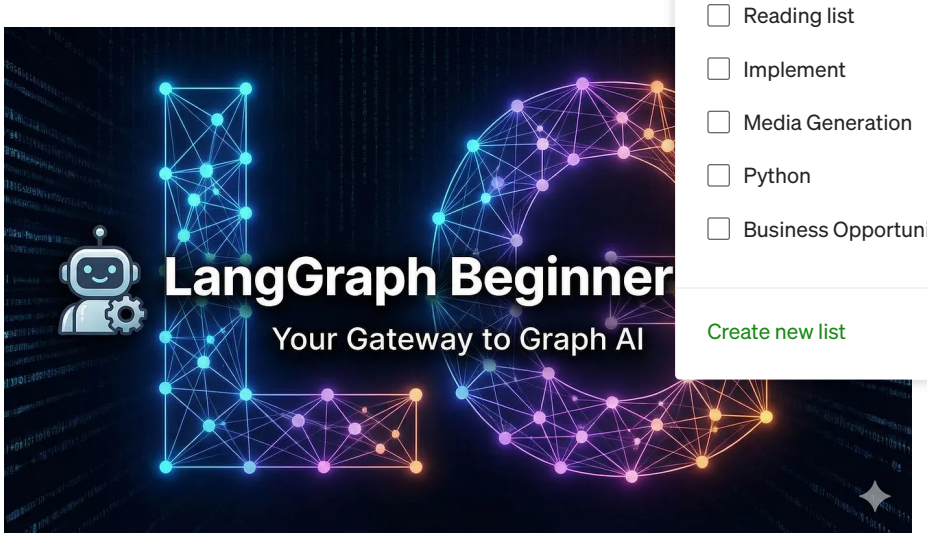
Activated Thinker

Member-only story

LangGraph Beginner Edition

You'll never need to google it again

Shreyas Naphad Follow 4 min read · Dec 19, 2025



AI-Generated Image

👉*For Non-Members link to the full story*

> *Stop! Just stop adding LangGraph tutorials to your to-do lists and never go back to them.*
> *This one article is designed to be read* ***once completely*** *and remembered forever.*

The honest problem with LangGraph

Most of the tutorials that I have seen explain **how to use LangGraph** but almost none explain **why it exists**.

So a lot of us end up trying to memorize code, copy-paste nodes and edges and forget everything in 2 weeks.

This article fixes that.

By the end:

- You'll understand **what LangGraph really is**
- Why it exists even though LangChain already exists
- How to think in **graphs, not prompts**
- And you'll build a **mini project** that makes LangGraph stick in your head permanently

Let's start from scratch.

What problem does LangGraph solve?

I want you to imagine a normal LLM app.

Let's say it looks like:

```
User → Prompt → LLM → Answer
```

That's fine for **simple chatbots.** But real apps are messy.

Real AI apps look like this:

- Ask a question
- Decide what to do
- Maybe search
- Maybe ask a follow-up
- Maybe loop until confident
- Maybe branch differently next time

Example:

> *"If question is unclear → ask clarification*
> *If question needs data → search*
> *If answer is good → stop*
> *Else → improve answer and try again"*

This is **not a straight line.** This is **logic + decisions + loops.**

👉 This is where **LangGraph exists.**

One-line definition (remember this)

> **LangGraph is a framework that lets you build LLM workflows as graphs instead of linear chains.**

That's it. There's no better definition than this.

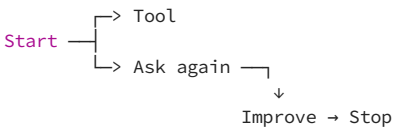
Chains vs Graphs (the key mental model)

LangChain (Chain)

```
Step 1 → Step 2 → Step 3 → Done
```

There's a fixed order here with no loops or branching. There is no memory of state logic.

LangGraph (Graph)



This can loop or have different branches. Can **decide next step and stop conditionally.**

> *If LangChain is a **pipeline**,*
> *LangGraph is a **flowchart with intelligence**.*

The core idea: State

This is the most important concept.

What is "state"?

State = shared memory passed between steps

Example state:

```
{
  "question": "What is RAG?",
  "answer": "",
  "confidence": 0.4,
  "tries": 1
}
```

Every step:

- reads from state
- updates state
- passes it forward

👉 You don't pass variables manually
👉 LangGraph manages the state flow

LangGraph has only 5 things to remember

If you remember these, you remember LangGraph forever.

1️⃣ State (the brain)

A Python class that **defines what data flows.**

```
from typing import TypedDict

class GraphState(TypedDict):
    question: str
    answer: str
    tries: int
```

Think:

> *"What information should all steps see?"*

2️⃣ Nodes (the workers)

A node is **just a function.**

```
def generate_answer(state):
    return {
        "answer": "LLM output here",
        "tries": state["tries"] + 1
    }
```

Important to remember

- Input = state
- Output = partial state update

3️⃣ Edges (the paths)

Edges decide **where to go next.**

```
graph.add_edge("generate", "evaluate")
```

Meaning:

> *"After generate, go to evaluate"*

4️⃣ Conditional edges (the decision-maker)

This is where LangGraph becomes powerful.

```
def should_continue(state):
    if state["tries"] >= 2:
        return "end"
    return "generate"
```

Now your graph can **think**. Based on a condition it will take the next step.

5️⃣ Entry point & END

```
graph.set_entry_point("generate")
```

END means:

> "Stop execution"

How LangGraph actually runs (important intuition)

LangGraph does this internally:

1. Start at the entry node
2. Pass state to node
3. Update state
4. Decide next edge
5. Repeat
6. Stop at END

It's literally a **state machine for LLMs**.

Mini Project: Self-improving Answer Bot (very simple)

This project alone will make LangGraph click.

Goal

- Generate an answer
- Check if it's "good enough"
- If not → improve
- Stop after 2 tries

Step 1: Define state

```
class GraphState(TypedDict):
    question: str
    answer: str
    tries: int
```

Step 2: Generate answer node

```
def generate(state):
    answer = f"Attempt {state['tries']}: This is a simple answer."
    return {
        "answer": answer,
        "tries": state["tries"] + 1
    }
```

Step 3: Decide whether to continue

```
def decide(state):
    if state["tries"] >= 2:
        return "end"
    return "generate"
```

Step 4: Build the graph

```
from langgraph.graph import StateGraph, END

graph = StateGraph(GraphState)
graph.add_node("generate", generate)
graph.set_entry_point("generate")
graph.add_conditional_edges(
    "generate",
    decide,
    {
        "generate": "generate",
        "end": END
    }
)
app = graph.compile()
```

Step 5: Run it

```
result = app.invoke({
    "question": "What is LangGraph?",
    "answer": "",
    "tries": 0
})

print(result)
```

What just happened (important)

- Graph started at `generate`
- Updated state
- Decision checked `tries`
- Loop ran again
- Stopped automatically

> *No loops written manually. No multiple if-else. Graph handled logic.*

Why LangGraph is loved by GenAI engineers

LangGraph is used when you need:

- Multi-step reasoning
- Tool + LLM coordination
- Agent loops
- Retry logic
- Decision-based workflows

This is **production AI**, not demos.

Data Science | Artificial Intelligence | Programming | Productivity | Machine Learning

Published in Activated Thinker

36K followers · Last published 3 hours ago

Follow

You have the thought, but you need to turn it on.

Written by Shreyas Naphad

7.7K followers · 38 following

Follow

Tech and sports enthusiast with a knack for combining skills like AI, machine learning, and creativity. I enjoy sharing what I learn and connecting with others.